

Overview

This specification adds a `/spec jtbd <specname>` slash command that performs a Jobs-To-Be-Done analysis of an existing spec's `SPEC.md` and writes the result to a `JTBD.md` file in the same spec folder, using an LLM-backed agent to extract the underlying "jobs" the feature is hired to do.

Jobs

Job 1 — Generate a JTBD analysis from a spec

- **Job statement:** When I have a written specification, I want to automatically extract the jobs the feature is hired to do, so I can understand the product's value proposition through the JTBD lens without manually re-reading and interpreting every requirement.
- **Job type:** Functional
- **Performer:** A developer or product analyst using ragent who authors or reviews specifications.
- **Related requirements:** FR-002, FR-006, FR-007, FR-013
- **Success signals:** A `JTBD.md` file appears in the target spec folder containing well-formed markdown with job statements following the "When ..., I want to ..., so I can ..." grammar, each job classified by type, and each job tracing to specific `FR-NNN/NFR-NNN` identifiers.

Job 2 — Protect existing analysis from accidental overwrite

- **Job statement:** When I have already generated a `JTBD.md` for a spec, I want the system to refuse to overwrite it unless I explicitly opt in, so I can avoid losing my prior analysis work through a careless re-run.
- **Job type:** Functional
- **Performer:** A developer or product analyst who has previously run `/spec jtbd` on the same spec.
- **Related requirements:** FR-003, FR-004
- **Success signals:** Re-running `/spec jtbd <specname>` without `--force` surfaces a status message that the file exists; re-running with `--force` overwrites the file atomically with a fresh, well-formed document.

Job 3 — Choose which agent performs the analysis

- **Job statement:** When I dispatch a JTBD analysis, I want to optionally direct it to a specific named agent, so I can control which LLM persona or capability set interprets my spec.
- **Job type:** Functional
- **Performer:** A developer who has configured custom agents and wants a particular one to author the analysis.
- **Related requirements:** FR-005
- **Success signals:** `/spec jtbd <specname> --agent <name>` dispatches to the named agent; supplying an unknown agent name produces an error and no task is spawned.

Job 4 — Get clear, actionable feedback on errors

- **Job statement:** When I invoke `/spec jtbd` with an invalid or missing spec name, I want to see an actionable error message naming the problem, so I can correct my command without guessing what went wrong.

- **Job type:** Functional
- **Performer:** A developer typing slash commands in the TUI who may mistype a spec name or reference a non-existent spec.
- **Related requirements:** FR-008, FR-009
- **Success signals:** An unknown spec name, invalid `SpecId`, missing `SPEC.md`, or unreadable `SPEC.md` each produce a specific error message naming the missing path, and no file is created or task spawned.

Job 5 — Discover and learn the command

- **Job statement:** When I am exploring the `/spec` command family, I want the help text and argument hints to mention `jtbd`, so I can discover and use the sub-command without reading external documentation.
- **Job type:** Functional
- **Performer:** A new or returning ragent user browsing available slash commands.
- **Related requirements:** FR-010, NFR-003
- **Success signals:** `/spec help` lists a row for `/spec jtbd [specname] [--force] [--agent <name>]`; the argument-hint table advertises the `jtbd` sub-command; `QUICKSTART.md` mentions the new command.

Job 6 — Trust the analysis is well-formed and traceable

- **Job statement:** When I read a generated `JTBD.md`, I want every job to follow the JTBD grammar and link back to specific requirement IDs, so I can trust the analysis is grounded in the spec rather than fabricated.
- **Job type:** Emotional
- **Performer:** A developer or product analyst reviewing the JTBD output for accuracy and completeness.
- **Related requirements:** FR-006, FR-007, FR-013
- **Success signals:** Every job uses the *"When <situation>, I want to <motivation>, so I can <expected outcome>"* grammar; each job lists related `FR-NNN/NFR-NNN` IDs or is explicitly marked *untraced*; re-runs produce a complete, well-formed document with no partial or merged content.

Job 7 — Observe and cancel in-flight analysis

- **Job statement:** When a JTBD analysis is running, I want to see its status in the TUI and be able to cancel it cleanly, so I can stay aware of what the system is doing and abort it if I change my mind.
- **Job type:** Functional
- **Performer:** A developer who dispatches the analysis and waits for it to complete.
- **Related requirements:** FR-011, FR-014
- **Success signals:** The TUI status line shows `"spec jtbd: <specname>"` during dispatch; a visible assistant message and `Info`-level log entry appear; cancelling the task terminates it and leaves no partially-written `JTBD.md` (or marks it `<!-- incomplete -->`).

Job 8 — Rely on a fast, non-disruptive, isolated dispatch path

- **Job statement:** When I run `/spec jtbd`, I want the command parsing and validation to be fast and free of unexpected side effects, so I can trust the system won't block the UI or make network calls I didn't authorise.

- **Job type:** Functional
- **Performer:** Any ragent user invoking the slash command.
- **Related requirements:** FR-001, FR-012, NFR-001, NFR-004
- **Success signals:** The parse-validate-prompt path completes within 50 ms on a warmed cache; no network I/O occurs outside the spawned agent task; all existing `/spec` sub-commands continue to parse and execute identically.

Out-of-Scope Jobs

- **JTBD analysis of non-SPEC.md files** — analysing `PLAN.md` or other documents is explicitly deferred (Out of scope, line 214).
- **Automatic regeneration on spec change** — keeping `JTBD.md` in sync when `SPEC.md` changes without an explicit re-run is deferred (Out of scope, line 215).
- **Server/CLI equivalent (`ragent spec jtbd`)** — a follow-up spec may add this; not covered here (Out of scope, line 217).
- **Multi-spec batch analysis (`/spec jtbd --all`)** — batch processing of multiple specs at once is deferred (Out of scope, line 218).
- **Template enforcement for LLM output** — the spec assumes LLM output quality is acceptable without a dedicated validation pass beyond markdown well-formedness (Risks and Assumptions, line 225).

Coverage Matrix

Requirement	Job(s) Supported
FR-001	Job 8
FR-002	Job 1
FR-003	Job 2
FR-004	Job 2
FR-005	Job 3
FR-006	Job 1, Job 6
FR-007	Job 1, Job 6
FR-008	Job 4
FR-009	Job 4
FR-010	Job 5
FR-011	Job 7
FR-012	Job 8
FR-013	Job 1, Job 6
FR-014	Job 7
NFR-001	Job 8

Requirement	Job(s) Supported
NFR-002	<i>unmapped</i>
NFR-003	Job 5
NFR-004	Job 8